

Лабораторная работа № 8-9

Тема: Программирование с использованием файлового ввода и вывода в языке Java.

Цель: Изучить классы, содержащиеся в пакете java.io, освоить потоковый ввод-вывод Java, научиться работать с файлами.

Краткая теория

В пакете java.io содержатся классы организующие ввод и вывод в Java, а также работу с файлами. Операции ввода и вывода в языке Java (как и в языке C++) основываются на понятии поток. Поток – источник или приемник информации.

Изначально в языке Java 1.0 присутствовали только байтовые потоки, базовые операции которых работали с байтами (тип **byte**). Позднее в версии Java 1.1 появились символьные потоки, базовые операции которых работают с символами (тип **char**).

На основе байтовых потоков созданы классы бинарного (двоичного) ввода и вывода, а на основе символьного потока – классы текстового ввода и вывода.

Класс File используется для работы непосредственно с файлами и файловой системой. Данный класс нельзя использовать для организации операций чтения или записи файлов. Конструкторы класса:

```
File(String dirPath)
File(String dirPath, String filename)
File(File dirObj, String filename)
```

- dirPath – имя пути файла,
- filename – имя файла или подкаталога,
- dirObj – объект File, указывающий на каталог.

В классе File определено большое количество методов для работы с файлами, некоторые из этих методов:

long length()	Возвращает длину файла
String getName()	Возвращает имя файла
String getParent()	Возвращает имя родительского каталога
String getPath()	Возвращает путь к файлу
boolean isDirectory()	Проверяет, является ли файл каталогом
boolean isFile()	Проверяет, является ли файл файлом
boolean isAbsolute()	Проверяет, является имя файла абсолютным
boolean isHidden()	Проверяет, является ли файл скрытым
boolean canExecute()	Проверяет, является ли файл выполняемым
boolean canRead()	Проверяет, является ли файл доступным для чтения
boolean canWrite()	Проверяет, является ли файл доступным для записи
boolean setExecutable(boolean val)	Устанавливает признак возможности выполнения файла
boolean setReadable(boolean val)	Устанавливает признак возможности чтения файла
boolean setWritable(boolean val)	Устанавливает признак возможности записи в файл
long lastModified()	Возвращает время модификации файла (в

	миллисекундах с 1.01.1970)
<code>boolean setLastModified(long time)</code>	Устанавливает время модификации файла (в миллисекундах с 1.01.1970)
<code>boolean exists()</code>	Проверяет: существует ли файл
<code>boolean delete()</code>	Удаляет файл или пустой каталог
<code>boolean rename(File newName)</code>	Переименовывает файл
<code>long getFreeSpace()</code>	Возвращает количество свободного места на томе
<code>long getTotalSpace()</code>	Возвращает размер тома
<code>long getUsableSpace()</code>	Возвращает количество используемого места на томе
<code>String [] list()</code>	Возвращает список файлов и каталогов
<code>String [] list(FilenameFilter fltr)</code>	Возвращает список файлов и каталогов согласно фильтру
<code>File [] listFiles()</code>	Возвращает список файлов и каталогов
<code>File [] listFiles(FilenameFilter fltr)</code>	Возвращает список файлов и каталогов согласно фильтру
<code>File [] listFiles(FileFilter fltr)</code>	Возвращает список файлов и каталогов согласно фильтру
<code>boolean mkdir()</code>	Создает каталог
<code>boolean mkdirs()</code>	Создает каталог и путь к нему

Базовыми интерфейсами потоков являются интерфейсы `Closeable` и `Flushable`. Объекты класса, реализующего `Closeable`, могут быть закрыты. Для этого они определяют метод:

```
void close() throws IOException
```

Этот метод закрывает поток, освобождая все ресурсы, которые тот мог удерживать.

Объекты класса, реализующего `Flushable`, могут заставить буферизированный вывод записываться в поток, к которому присоединен этот поток. Для этого они определяют метод:

```
void flush () throws IOException
```

Сброс потока обычно вынуждает буферизированный вывод физически записываться на устройство.

Базовые потоки:

- `InputStream` – байтовый поток ввода.
- `OutputStream` – байтовый поток вывода.
- `Reader` – символьный поток ввода.
- `Writer` – символьный поток вывода.

Класс `InputStream` – абстрактный класс, определяющий байтовый поток ввода. Реализует интерфейс `Closeable`. Большинство методов класса генерируют `IOException`. Методы класса:

int available()	Возвращает количество байт ввода, доступных в данный момент для чтения
void close()	Закрывает поток

void mark(int numBytes)	Помещает метку в текущую точку потока, которая остается корректной, пока не будет прочитано numBytes байт из потока
boolean markSupported()	Проверяет, поддерживается ли метка
int read()	Читает байт из потока (если поток пуст - -1)
int read(byte buffer[])	Читает байты из потока в buffer, возвращает фактически прочитанное количество байт
int read(byte buffer[], int offset, int numBytes)	Читает байты из потока в buffer, начиная с buffer[offset] и возвращает фактически прочитанное количество байт
void reset()	Возвращается к метке установленной mark
long skip(long numBytes)	Пропускает numBytes потока, возвращает количество действительно проигнорированный байт потока

Класс `OutputStream` – абстрактный класс, определяющий байтовый поток вывода. Реализует интерфейсы `Closeable` и `Flushable`. Большинство методов класса генерируют `IOException`. Методы класса:

int close()	Закрывает поток
void flush()	Очищает все буферы вывода, записывая их
void write(int b)	Записывает младший байт b в поток
void write(byte buffer[])	Записывает buffer в поток
void write(byte buffer[], int numBytes)	Записывает первых numBytes байт из buffer в поток

Класс `FileInputStream` создает `InputStream`, который можно использовать для чтения потока байт из файла. Конструкторы:

```
FileInputStream(String filePath)
FileInputStream(File fileObj)
```

Класс `FileOutputStream` создает `OutputStream`, который можно использовать для записи потока байт в файл. Конструкторы:

```
FileOutputStream(String filePath)
FileOutputStream(File fileObj)
FileOutputStream(String filePath, boolean append)
FileOutputStream(File fileObj, boolean append)
```

Класс `ByteArrayInputStream` – реализация входного потока, использующего байтовый массив в качестве источника данных. Конструкторы:

```
ByteArrayInputStream(byte array[])
ByteArrayInputStream(byte array[], int start, int numBytes)
```

Класс `ByteArrayOutputStream` – реализация выходного потока, использующего байтовый массив в качестве места назначения. Конструкторы:

```
ByteArrayOutputStream()
ByteArrayOutputStream(int numBytes)
```

По умолчанию начальный размер 32 байта. Получить содержимое можно методом `toByteArray()`.

Фильтруемые потоки байт – это оболочки вокруг входных или выходных потоков, которые прозрачно предоставляют дополнительный уровень функциональности. Эти потоки обычно доступны методам, которые ожидают обобщенного потока, являющегося суперклассом фильтрованного потока. Фильтруемые потоки байт – это классы `FilterOutputStream` и `FilterInputStream`. Их конструкторы:

```
FilterOutputStream(OutputStream os)
FilterInputStream(InputStream is)
```

Класс `BufferedInputStream` – буферизированный поток ввода. Конструкторы:

```
BufferedInputStream(InputStream is)
BufferedInputStream(InputStream is, int bufSize)
```

Класс `BufferedOutputStream` – буферизированный поток вывода. Конструкторы:

```
BufferedOutputStream(OutputStream is)
BufferedOutputStream(OutputStream is, int bufSize)
```

Класс `PushbackInputStream` позволяет возвращать данные обратно в поток ввода.

```
PushbackInputStream(InputStream is)
PushbackInputStream(InputStream is, int bufSize)
```

Методы класса:

```
void unread(int ch)
void unread(byte buffer[])
void unread(byte buffer[], int offset, int numChars)
```

Класс `PrintStream` является одним из наиболее часто используемых потоков вывода в языке Java и реализует интерфейсы `Appendable`, `Closeable`, `Flushable`. Объект `System.out` является экземпляром этого класса.

```
PrintStream(OutputStream os)
PrintStream(OutputStream os, boolean freeOnNewLine)
PrintStream(OutputStream os, boolean freeOnNewLine, String charset)
PrintStream(File outFile) throws FileNotFoundException
PrintStream(File outFile, String charset) throws FileNotFoundException,
                                             UnsupportedEncodingException
PrintStream(String fileName) throws FileNotFoundException
PrintStream(String fileName, String charset) throws FileNotFoundException,
                                             UnsupportedEncodingException
```

Основные методы: `print()`, `println()`, `printf()`

Классы `DataOutputStream` (расширяет `FilterOutputStream`) и `DataInputStream` (расширяет `FilterInputStream`) позволяют читать и писать примитивные типы в байтовых потоках.

```
DataOutputStream(OutputStream os)
DataInputStream(InputStream is)
```

Данные классы поддерживают все методы базовых классов и добавляют свои. Класс `DataOutputStream` добавляет методы записи примитивных типов в байтовый поток:

```
final void writeDouble(double value) throws IOException
final void writeInt(int value) throws IOException
и др.
```

Класс `DataInputStream` добавляет методы чтения примитивных типов из байтового потока:

```
double readDouble() throws IOException
int readInt() throws IOException
и др.
```

Класс `RandomAccessFile` инкапсулирует класс произвольного доступа. Данный класс реализует интерфейсы `DataInput` и `DataOutput`, которые определяют базовые методы ввода-вывода, а также интерфейс `Closeable`.

```
RandomAccessFile(File fileObj, String mode) throws FileNotFoundException
RandomAccessFile(String fileName, String mode) throws FileNotFoundException
```

Параметр mode может принимать значения:

- «r» - чтение,
- «rw» - чтение и запись
- «rws» - чтение и немедленной записью

Дополнительные методы:

void seek(long pos) throws IOException

void setLength(long len) throws IOException

Символьные потоки в языке Java основаны на двух классах:

- Reader – абстрактный класс, определяющий символьный поток ввода в Java и реализующий интерфейсы Closeable и Readable. Большинство методов генерируют IOException при возникновении ошибки;
- Writer – абстрактный класс, определяющий символьный поток вывода в Java и реализующий интерфейсы Closeable, Flushable и Appendable.

Методы класса Reader:

abstract void close()	Закрывает поток
void mark(int numChars)	Помещает метку в текущую позицию, которая остается корректной до тех пор, пока не будет прочитано numChars символов
boolean markSupported()	Проверяет, поддерживает ли поток метку
int read()	Читает один символ из потока (-1 – конец файла)
int read(char buff[])	Читает символы в buff, возвращает количество прочитанных символов (<= buff.length)
int read(char buff[], int offset, int num)	Читает символы в buff с позиции offset и не более num символов, возвращает количество прочитанных символов
boolean ready()	Возвращает true, если следующий запрос не будет ждать
void reset()	Сбрасывает указатель ввода на ранее установленную метку
long skip(long numChars)	Пропускает numChars символов, возвращая количество действительно пропущенных символов

Методы класса Writer:

Writer append(char ch)	Добавляет ch в конец потока
Writer append(CharSequence seq)	Добавляет seq в конец потока
Writer append(CharSequence seq, int begin, int end)	Добавляет диапазон seq (с позиции begin до позиции end-1) в конец потока
abstract void close()	Закрывает поток
abstract void flush()	Очищает поток, записывая данные на устройство
void write(int ch)	Записывает один символ в поток
void write(char buff[])	Записывает buff в поток

abstract write(char buff[], int offset, int num)	Записывает num символов с позиции offset из buff в поток
void write(String str)	Записывает всю строку в поток
void write(String str, int offset, int num)	Записывает num символов с позиции offset из str в поток

Класс `FileReader` – класс, создающий `Reader` для чтения из файла.

```
FileReader(String fileName)
FileReader(File fileObj)
```

Класс `FileWriter` – класс, создающий `Writer` для записи в файл.

```
FileWriter(String fileName)
FileWriter(String fileName, boolean append)
FileWriter(File fileObj)
FileWriter(File fileObj, boolean append)
```

Класс `CharArrayReader` – класс, реализующий входной поток из символьного массива.

```
CharArrayReader(char array[])
CharArrayReader(char array[], int start, int len)
```

Класс `CharArrayWriter` – класс, реализующий выходной поток в символьный массив

```
CharArrayWriter()
CharArrayWriter(int len)
```

`len` – начальная длина буфера

Получить сформированный массив символов можно методом `toCharArray`.

Класс `BufferedReader` – реализует буферизированный ввод.

```
BufferedReader(Reader is)
BufferedReader(Reader is, int size)
```

Класс `BufferedWriter` – реализует буферизированный вывод.

```
BufferedWriter(Writer os)
BufferedWriter(Writer os, int size)
```

Класс `PushbackReader` позволяет помещать прочитанные символы обратно в поток.

```
PushbackReader(Reader is)
PushbackReader(Reader is, int size)
```

Дополнительные методы `PushbackReader`:

```
void unread(int ch)
void unread(byte buff[])
void unread(byte buff[], int offset, int len)
```

Класс `PrintWriter` – символ-ориентированная версия `PrintStream` и реализует интерфейсы `Appendable`, `Closeable`, `Flushable`. Конструкторы класса:

```
PrintWriter (OutputStream os)
PrintWriter (OutputStream os, boolean flushOnNewLine)
PrintWriter (Writer os)
PrintWriter (Writer os, boolean flushOnNewLine)
PrintWriter (File outFile) throws FileNotFoundException
PrintWriter (File outFile, String charset) throws FileNotFoundException,
    UnsupportedEncodingException
PrintWriter (String fileName) throws FileNotFoundException
PrintWriter (String fileName, String charset) throws FileNotFoundException,
    UnsupportedEncodingException
```

Основные методы: `print()`, `println()`, `printf()`, `format()`.

Сериализация – это процесс записи состояния в байтовый поток. Десериализация – процесс восстановления состояния объекта из потока. В Java эти процессы в основном полностью автоматизированы и реализуются определенным набором интерфейсов и классов.

Интерфейс `Serializable` не содержит никаких методов. Если класс реализует этот интерфейс, то объекты этого класса могут быть сериализованы и десериализованы. Переменные типа `transient` и `static` в процессе сериализации не сохраняются.

Интерфейс `Externalizable` предназначен для реализации возможности программисту самому управлять сериализацией и десериализацией объекта. Данный интерфейс описывает методы:

```
void readExternal(ObjectInput inStream) throws IOException
void writeExternal(ObjectOutput outStream) throws IOException
```

Интерфейсы `ObjectOutput` и `ObjectInput` расширяют интерфейсы `DataOutput` и `DataInput` соответственно.

Интерфейс `ObjectOutput` содержит байт-ориентированные методы: `close()`, `flush()`, `write()`, а также метод записи объекта:

```
void writeObject(Object obj)
```

Интерфейс `ObjectInput` содержит байт-ориентированные методы: `close()`, `available()`, `read()`, `skip()`, а также метод чтения объекта:

```
Object readObject()
```

Класс `ObjectOutputStream` расширяет класс `OutputStream` и реализует интерфейс `ObjectOutput`. Этот класс отвечает за запись объекта в поток.

```
ObjectOutputStream(OutputStream os) throws IOException
```

Данный класс наряду с методами записи байт в поток содержит реализацию метода `writeObject()`, а также методы записи всех примитивных типов:

```
void writeDouble(double val)
void writeInt(int val)
```

и др.

Класс `ObjectInputStream` расширяет класс `InputStream` и реализует интерфейс `ObjectInput`. Этот класс отвечает за чтение объекта из потока.

```
ObjectInputStream(InputStream is) throws IOException
```

Данный класс наряду с методами чтения байт из потока содержит реализацию метода `readObject()`, а также методы чтения всех примитивных типов:

```
double readDouble()
int readInt()
```

и др.

Ход работы

Задание 1

Реализовать программу согласно варианту заданий. Имя исходного файла или каталога пользователь вводит с клавиатуры. Варианты заданий:

1	Определить суммарный размер всех файлов определенного типа (расширения) в каталоге. Расширение указывает пользователь.
2	Подсчитать в каталоге и всех подкаталогах количество файлов, размер которых более N байт. Значение N вводит пользователь.
3	Удалить из каталога и всех подкаталогов все файлы с определенным расширением. Расширение указывает пользователь.
4	Найти в каталоге и всех подкаталогах файл определенного типа (расширения) с наибольшим размером. Расширение указывает пользователь.
5	Удалить из каталога и из каждого его подкаталога файл наименьшего размера.
6	Определить в каждом подкаталоге указанного каталога файл, время модификации которого наиболее позднее. Имена найденных файлов вывести на экран.
7	Вычислить суммарный объем файлов в каждом из подкаталогов заданного каталога.
8	Удалить подкаталог заданного каталога, в котором меньше всего файлов.

9	Вывести на экран имена всех подкаталогов заданного каталога, в которых встречается хотя бы один текстовый файл.
10	Вывести на экран список файлов заданного каталога, упорядочив их по размеру (направление указывает пользователь)
11	Создать пустой каталог «EMPTY» в подкаталоге заданного каталога, в котором нет текстовых файлов (расширение txt).
12	Вычислить полный суммарный объем всех файлов в указанном каталоге, включая все его подкаталоги.
13	Создать пустой файл «empty.txt» в каждом из подкаталогов исходного каталога, в которых вообще нет файлов.
14	Вывести на экран все файлы определенного типа (расширения) в каталоге. Расширение указывает пользователь.
15	Вывести на экран список файлов заданного каталога, упорядочив их по времени последней модификации (направление указывает пользователь)
16	Подсчитать в каталоге и всех подкаталогах количество файлов, размер которых менее N байт. Значение N вводит пользователь.
17	Создать пустой каталог «EMPTY» в подкаталоге заданного каталога, в котором наибольшее количество файлов.
18	Удалить из каталога и из каждого его подкаталога файл наибольшего размера.
19	Вывести на экран имена всех подкаталогов заданного каталога, в которых содержится ровно три файла.
20	Удалить из каталога и всех подкаталогов все файлы, размер которых лежит в диапазоне [N, M]. Диапазон указывает пользователь.
21	Удалить подкаталог заданного каталога, в котором суммарный объем файлов наименьший.
22	Найти в каталоге и всех его подкаталогах файл определенного типа (расширения) с наименьшим размером. Расширение указывает пользователь.
23	Вычислить полный суммарный объем всех текстовых файлов (расширение txt) в указанном каталоге, включая все его подкаталоги.
24	Определить в каждом подкаталоге указанного каталога файл, время модификации которого наиболее раннее. Имена найденных файлов вывести на экран.
25	Вычислить количество файлов в каждом из подкаталогов заданного каталога.

Задание 2

Разработать программу обработки текстовых файлов согласно варианту задания.

Варианты заданий:

1	Файл содержит произвольный текст, в котором встречаются целые числа, заключенные в скобки (круглые, квадратные и фигурные). Определить сумму этих чисел и записать числа заключенные в круглые скобки в файл 1.txt, в квадратные скобки – 2.txt, фигурные скобки – 3.txt.
2	Файл содержит целые числа. Удалить из файла все числа, кратные N (вводится пользователем).
3	Даны два текстовых файла, содержащих целые числа. Числа в каждом файле различны. Сформировать третий файл, записав в него все значения, встречающиеся только в одном из исходных файлов. Имя результирующего файла – result.txt.
4	Файл содержит вещественные числа. Удалить из файла все числа, меньшие среднего значения всех чисел в файле.
5	Файл содержит целые числа. Переписать файл, упорядочив числа по следующему принципу: сначала идут нулевые значения, затем положительные значения в порядке убывания, а затем отрицательные числа в порядке возрастания.
6	Файл содержит слова. Продублировать все слова в файле, в которых встречаются три одинаковых буквы.

7	Файл содержит вещественные числа. Если положительных чисел в файле больше чем отрицательных, то удалить из файла все отрицательные числа. В противном случае – все положительные.
8	Файл содержит целые числа. После каждой серии в файле вставить нулевое значение. Серия – последовательность одинаковых значений.
9	Файл содержит произвольный текст. Составить словарь файла, в каждой строке содержится слово и количество раз, которое оно встречается в исходном файле. Имя файла-словаря – words.txt.
10	Файл содержит слова, целые числа и вещественные числа. Записать все слова в алфавитном порядке в файл words.txt, целые числа, упорядоченные по возрастанию, в файл integers.txt, вещественные числа в порядке убывания в файл floats.txt.
11	Файл содержит целые числа. Заменить все числа, представляющие число три в целочисленной степени сочетанием 3^n (где n – степень). Например: $9 - 3^2$, $27 - 3^3$.
12	В каждой строке файла содержится ряд вещественных значений. Создать новый текстовый файл, в каждой строке которого содержится минимальное, максимальное и среднее значение из каждой строки исходного файла.
13	В файле содержатся слова. Удалить из файла все повторяющиеся слова.
14	Дан файл, содержащий произвольный текст внутри которого встречаются времена в формате чч.мм.сс. Записать все времена в файл times.txt в хронологическом порядке.
15	Файл содержит произвольный текст, в котором встречаются слова, заключенные в двойные или одинарные кавычки. Записать эти слова без кавычек в файл result.txt сначала слова в одинарных кавычках, в алфавитном порядке, а затем слова в двойных кавычках, в обратном алфавитном порядке.
16	Файл содержит слова. Удалить из файла все слова, длина которых больше средней длины всех слов в файле.
17	В каждой строке файла содержится имя текстового файла, находящегося в текущем каталоге и содержащегося целые числа. Переписать исходный файл, записав после каждого имени файла среднее значение его элементов.
18	Файл содержит вещественные числа. Перед каждым пятым положительным числом вставить нулевое значение (0.0).
19	Даны два текстовых файла, содержащих целые числа. Числа в каждом файле различны. Сформировать третий файл, записав в него все значения, встречающиеся как в первом так и во втором файле. Имя результирующего файла – result.txt.
20	Файл содержит вещественные числа. Записать все положительные числа в порядке возрастания в файл positive.txt, а отрицательные числа в порядке убывания в файл negative.txt.
21	Дан текстовый файл, содержащий целые числа. Переписать файл, удалив из него наиболее часто встречающееся значение. Если таких значений несколько, то только последнее из них.
22	Файл содержит слова. Заключить слова, начинающиеся с согласной буквы в двойные кавычки.
23	Файл содержит целые числа. Удалить из файла все числа, число бит установленных в единицу, в которых более трех.
24	Дан файл, содержащий произвольный текст внутри которого встречаются даты в формате дд.мм.гггг. Записать все даты в файл dates.txt в хронологическом порядке.
25	Файл содержит слова и целые числа. Удалить из файла все слова, а целые числа переписать в следующем порядке: сначала в порядке возрастания положительные числа, затем нулевые значения и, в конце, отрицательные числа в порядке убывания.

Задание 3

Разработать программу обработки бинарных файлов согласно варианту задания. После обработки содержимое файла результата необходимо вывести на экран. Варианты заданий:

1	Дан файл, содержащий целые числа (тип short). Удалить из файла все числа кратные числу N (вводится пользователем).
2	Дан файл, содержащий целые числа (тип long). Увеличить все серии минимальной длины в два раза.
3	Дан файл, содержащий вещественные числа (тип double). Удалить из файла все числа, расположенные между максимальным и минимальным значениями.
4	Файл содержит целые числа (тип int). Упорядочить положительные числа в файле, а отрицательные числа удалить. Направление сортировки выбирает пользователь.
5	Дан файл, содержащий целые числа (тип short). Удалить из файла все простые числа.
6	Файл содержит вещественные числа (тип double). Увеличить в два раза все значения меньшие среднего арифметического значения всего файла.
7	Дан файл, содержащий вещественные числа (тип float). Упорядочить в файле числа, расположенные между максимальным и минимальным значениями. Направление выбирает пользователь.
8	Файл содержит вещественные числа. Уменьшить в два раза все значения большие среднего арифметического значения всего файла.
9	Файл содержит вещественные числа (тип double). Переставить числа в файле так, чтобы сначала располагались положительные значения, а затем отрицательные. Порядок следования самих чисел не менять.
10	Файл содержит целые числа (тип int). Найти в файле возрастающую последовательность значений максимальной длины и записать ее в конец файла. Если таких последовательностей несколько, то только первую из них.
11	Дан файл, содержащий целые числа (тип short). Перед каждой серией в файле вставить нулевое значение.
12	Файл содержит вещественные числа (тип float). Удалить из файла все локальные минимумы. Удалять только те минимумы, которые содержатся в исходном файле, а не появляются в процессе обработки.
13	Дан файл, содержащий вещественные числа (тип double). Перед каждой возрастающей последовательностью значений в файле вставить нулевое значение.
14	Дан файл, содержащий целочисленные значения (тип byte). Переписать в обратном порядке все значения кратные N. Остальные значения не изменять.
15	Дан файл, содержащий целые числа (тип long). Увеличить каждую серию длины N (вводится пользователем) на один элемент.
16	Файл содержит целые числа (тип int). Удалить из файла значение, встречающееся наибольшее количество раз. Если таких значений несколько, то минимальное из них.
17	Дан файл, содержащий целые числа (тип short). Удалить из файла все серии, длина которых кратна числу N (вводится пользователем).
18	Даны два файла, содержащих упорядоченные по возрастанию вещественные значения (тип double). Сформировать третий файл, записав туда значения из первых двух не нарушая упорядоченности. Сортировку использовать нельзя.
19	Дан файл, содержащий целые числа (тип int). Поменять местами серии максимальной и минимальной длины. Если таких серий несколько, то последние из них.
20	Даны два файла, содержащих целые числа (тип short). Удалить из первого файла, все значения встречающиеся во втором файле.
21	Дан файл, содержащий вещественные числа (тип double). Перед каждым третьим положительным значением вставить предыдущее значение в файле.
22	Дан файл, содержащий вещественные числа (тип float). Упорядочить числа в файле по возрастанию или убыванию их дробной части. Направление выбирает пользователь.

23	Дан файл, содержащий целые числа (тип long). Уменьшить серию максимальной длины в файле в два раза. Если таких серий несколько, то только первую из них. Серии единичной длины не изменять.
24	Два файла содержат целочисленные значения (тип int). Сформировать третий файл, записав в него те значения, которые встречаются как в первом, так и втором файлах. Не допускать дублирующихся значений.
25	Дан файл, содержащий вещественные числа (тип float). Продублировать все элементы, расположенные между максимальным и минимальным значениями в файле.

Контрольные вопросы

1. Что такое поток?
2. Какие виды потоков бывают в Java.
3. Какие классы реализуют базовые потоки в Java?
4. Перечислите методы класса InputStream.
5. Перечислите методы класса OutputStream.
6. Какие классы используются для байтового ввода и вывода из файлов в Java?
7. Какие классы используются для байтового ввода и вывода из массивов в Java?
8. Какие классы используются для буферизированного байтового ввода и вывода в Java?
9. Какой класс используется для произвольного доступа к содержимому файла?
10. Перечислите методы класса Reader.
11. Перечислите методы класса Writer.
12. Какие классы используются для символьного ввода и вывода из файлов в Java?
13. Какие классы используются для символьного ввода и вывода из массивов в Java?
14. Какие классы используются для буферизированного байтового ввода и вывода в Java?
15. Что такое сериализация?
16. Какие интерфейсы описывают сериализацию в языке Java?
17. Какие классы реализуют сериализацию в языке Java?